

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 01

SMS SPAM DETECTION USING NAIVE BAYES CLASSIFIER

COURSE CODE: AI 484

COURSE TITLE: ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Suman Saha

Assistant Professor

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

OCTOBER 13, 2026

1. Lab Problem & Objectives

Problem Statement:
Given the SMS Spam Collection dataset, how can you use the Naive Bayes classifier to automatically detect whether an incoming SMS is spam or not?

Key Objectives & Assigned Tasks:

1. **Dataset Acquisition & Loading:** Load the standard SMS Spam Collection dataset into a structured tabular format.
2. **Text Preprocessing & Vectorization:** Clean textual content (lowercasing, punctuation and numerical removal) and extract numerical representations via Count Vectorization (Bag-of-Words).
3. **Model Formulation & Training:** Train a Multinomial Naive Bayes classifier ($MultinomialNB$) on stratified train-test splits to learn statistical boundaries separating spam from ham.
4. **Performance Evaluation:** Evaluate the trained model on unseen test samples and determine Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix.
5. **Novel Sample Inference:** Predict whether the incoming message:

   "Congratulations! You have won a free ticket. Call now!"

   is classified as spam or ham, along with posterior confidence scores.

GitHub Repository Link:
https://github.com/Masud744/AI-ML-Lab/tree/main/Lab_01

2. Mathematical Formulation of Naive Bayes

The Naive Bayes classifier is a probabilistic machine learning algorithm grounded in **Bayes' Theorem**. Given an SMS message represented as a vector of word tokens $\mathbf{x} = (w_1, w_2, \ldots, w_n)$, the posterior probability of the message belonging to class $C_k \in \{\text{ham}, \text{spam}\}$ is given by:

$$P(C_k \mid \mathbf{x}) = \frac{P(C_k) \cdot P(\mathbf{x} \mid C_k)}{P(\mathbf{x})} \tag{1}$$

Assuming the conditional independence of words given the class label:

$$P(\mathbf{x} \mid C_k) = \prod_{i=1}^{n} P(w_i \mid C_k) \tag{2}$$

The classification decision rule assigns the class that maximizes the posterior likelihood:

$$\hat{y} = \arg\max_{C_k} P(C_k) \prod_{i=1}^{n} P(w_i \mid C_k) \tag{3}$$

To avoid the zero-frequency problem where an unseen word forces the entire likelihood product to zero, **Laplace Smoothing** ($\alpha = 1$) is employed:

$$P(w_i \mid C_k) = \frac{N_{k,i} + \alpha}{N_k + \alpha \cdot |V|} \tag{4}$$

where $N_{k,i}$ is the count of word w_i in class C_k, N_k is the total count of words in class C_k, and $|V|$ denotes the vocabulary size.

3. Methodology & Implementation Architecture

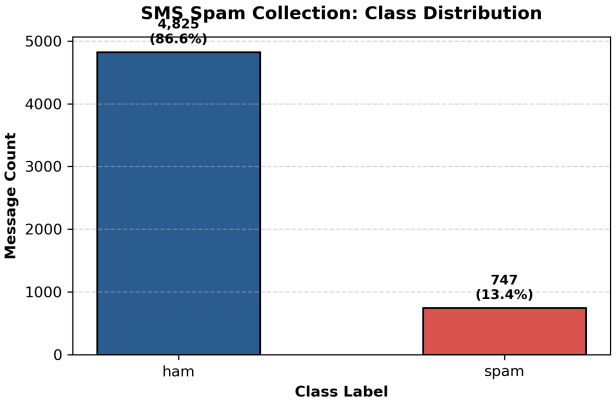


Figure 1: SMS Class Distribution ($N = 5,572$)

Dataset Exploration:

The SMS Spam Collection consists of $5,572$ tagged SMS messages. The distribution reflects standard real-world imbalance:

- **Legitimate (Ham):** $4,825$ messages (86.6%)
- **Unsolicited (Spam):** 747 messages (13.4%)

Pipeline Stages:

1. **Text Normalization:** Converting messages to lowercase, stripping punctuation via translation tables, and eliminating numeric digits.
2. **Stratified Splitting:** An $80\% - 20\%$ train-test split ($4,457$ training records, $1,115$ test records) maintaining identical class proportions across both splits.
3. **Bag-of-Words Vectorization:** Tokenizing corpus with `CountVectorizer`, resulting in a feature vocabulary of $7,537$ distinct unigrams.
4. **Model Training:** Fitting `MultinomialNB(alpha=1.0)`.

4. Experimental Results & Model Evaluation

The trained classifier was evaluated on $1,115$ unseen test messages (966 ham, 149 spam). The experimental metrics confirm strong discriminative capability:

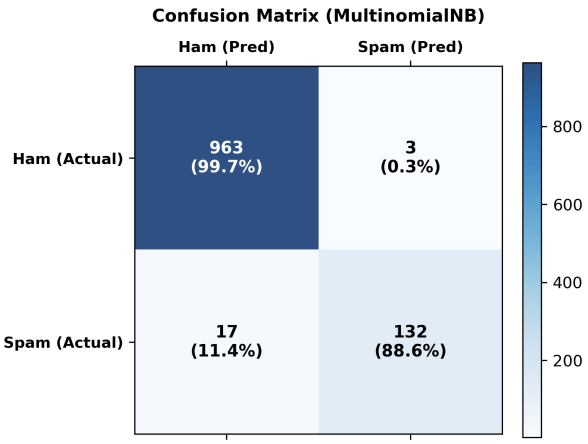


Figure 2: Confusion Matrix on Test Set

Performance Summary Table:

Metric	Spam Class	Overall
Accuracy	–	**98.21%**
Precision	**97.78%**	98.00%
Recall	**88.59%**	94.00%
F1-Score	**92.96%**	96.00%

Observations on Misclassification:

- **False Positives:** Only 3 legitimate messages (0.31%) were incorrectly flagged as spam, demonstrating exceptional precision (97.78%).
- **False Negatives:** 17 spam messages were missed due to short length or colloquial phrasing lacking typical spam keywords.

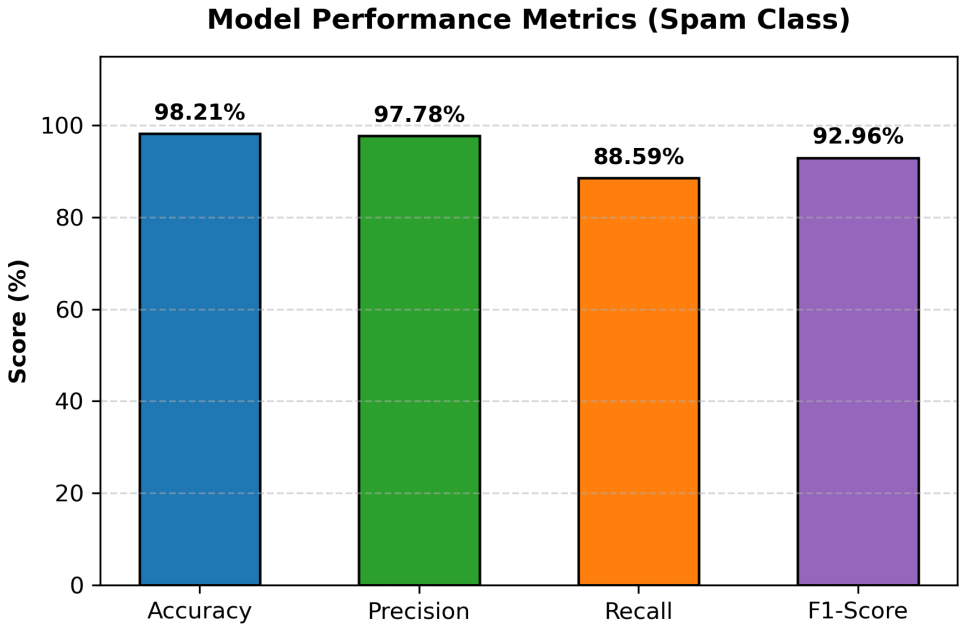


Figure 3: Performance Metrics Summary for Spam Detection

5. Inference on Assigned Test Query

The laboratory task required testing the model against the unseen prompt:

"Congratulations! You have won a free ticket. Call now!"

Execution & Prediction Pipeline:

1. **Raw Input:** `"Congratulations!  You have won a free ticket.  Call now!"`
2. **Cleaned Token Sequence:** `"congratulations you have won a free ticket call now"`
3. **Vectorized Representation:** Transformed through the fitted vocabulary, matching high-frequency spam trigger unigrams: `"congratulations"`, `"won"`, `"free"`, `"ticket"`, `"call"`, `"now"`.
4. **Model Output Class:** SPAM
5. **Class Probability Distribution:**
   - $P(\text{Spam} \mid \text{Message}) = \mathbf{99.9868}\%$
   - $P(\text{Ham} \mid \text{Message}) = \mathbf{0.0132}\%$

Result Analysis:
The high concentration of marketing, prize solicitation, and urgency terms yielded a conditional probability overwhelmingly favoring the spam class ($> 99.98\%$). The model successfully and decisively identified the malicious nature of the message.

6. Discussion & Conclusion

- **Algorithmic Efficiency:** Multinomial Naive Bayes demonstrates exceptional computational efficiency during both training and inference, completing feature transformations and model fitting across over $5,500$ records in sub-second intervals.
- **Precision vs. Recall Trade-off:** In SMS spam filtering, precision is paramount; false positives (labeling critical legitimate messages as spam) are significantly more detrimental to user experience than occasionally allowing spam through. Achieving a 97.78% spam precision satisfies this practical engineering requirement.
- **Conclusion:** The experiment validated the efficacy of the Naive Bayes algorithm in processing textual data for binary classification. The pipeline successfully ingested the SMS Spam Collection dataset, completed end-to-end preprocessing, achieved an overall accuracy of 98.21%, and correctly flagged the test query with extreme certainty.

7. Python Source Code Listing (spam_classifier.py)

GitHub Repository File:
https://github.com/Masud744/AI-ML-Lab/blob/main/Lab_01/src/spam_classifier.py

```python
#!/usr/bin/env python3
"""
Lab 01: SMS Spam Detection using Naive Bayes Classifier
Course: AI 484 - Artificial Intelligence and Machine Learning Sessional
Department of Internet of Things and Robotics Engineering (IRE)
University of Frontier Technology, Bangladesh (UFTB)

Author: Shahriar Alom Masud
Student ID: 2201032
Session: 2022-2023
"""

import os
import re
import string
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB


# ============================================================================
# 1. TEXT PREPROCESSING FUNCTION
# ============================================================================
def clean_text(text: str) -> str:
    """Preprocesses input text:

    - Converts to lowercase
    - Removes punctuation and digits
    - Removes excessive whitespaces
    """
    text = text.lower()
    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))
    # Remove numbers / digits
    text = re.sub(r"\d+", "", text)
    # Remove extra spaces
    text = re.sub(r"\s+", " ", text).strip()
    return text


# ============================================================================
# 2. DATASET LOADER
# ============================================================================
def load_dataset(filepath: str) -> pd.DataFrame:
    """Loads the SMS Spam Collection dataset from tab-separated file."""
    if not os.path.exists(filepath):
        raise FileNotFoundError(f"Dataset not found at {filepath}")

    df = pd.read_csv(filepath, sep="\t", header=None, names=["label", "message"])
    return df


# ============================================================================
# 3. VISUALIZATION EXPORTERS
# ============================================================================
def export_visualizations(df, y_test, y_pred, output_dir: str):
    """Exports high-resolution figures for LaTeX lab report."""
    os.makedirs(output_dir, exist_ok=True)

    # 1. Class Distribution Plot
```

```
    fig, ax = plt.subplots(figsize=(6, 4), dpi=300)
    counts = df["label"].value_counts()
    colors = ["#2b5c8f", "#d9534f"]
    bars = ax.bar(counts.index, counts.values, color=colors, width=0.5, edgecolor="black", linewidth
        =1.2)
    ax.set_title("SMS Spam Collection: Class Distribution", fontsize=12, fontweight="bold", pad=12)
    ax.set_ylabel("Message Count", fontsize=10, fontweight="bold")
    ax.set_xlabel("Class Label", fontsize=10, fontweight="bold")
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    # Annotate bar counts and percentages
    total = len(df)
    for bar in bars:
        h = bar.get_height()
        pct = (h / total) * 100
        ax.annotate(
            f"{h:,}\n({pct:.1f}%)",
            xy=(bar.get_x() + bar.get_width() / 2, h),
            xytext=(0, 4),
            textcoords="offset points",
            ha="center",
            va="bottom",
            fontsize=9,
            fontweight="bold",
        )
    plt.tight_layout()
    dist_path = os.path.join(output_dir, "class_distribution.png")
    fig.savefig(dist_path, bbox_inches="tight")
    plt.close(fig)
    print(f"[+] Saved class distribution plot: {dist_path}")

    # 2. Confusion Matrix Plot
    cm = confusion_matrix(y_test, y_pred, labels=["ham", "spam"])
    fig, ax = plt.subplots(figsize=(5.2, 4.5), dpi=300)
    cax = ax.matshow(cm, cmap="Blues", alpha=0.85)

    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            val = cm[i, j]
            pct = (val / np.sum(cm[i, :])) * 100
            color = "white" if val > cm.max() / 2 else "black"
            ax.text(j, i, f"{val}\n({pct:.1f}%)", ha="center", va="center", color=color, fontsize
                =10.5, fontweight="bold")

    fig.colorbar(cax, fraction=0.046, pad=0.05)
    ax.set_xticks([0, 1])
    ax.set_yticks([0, 1])
    ax.set_xticklabels(["Ham (Pred)", "Spam (Pred)"], fontsize=9.5, fontweight="bold")
    ax.set_yticklabels(["Ham (Actual)", "Spam (Actual)"], fontsize=9.5, fontweight="bold")
    ax.set_title("Confusion Matrix (MultinomialNB)", fontsize=11, fontweight="bold", pad=12)
    plt.tight_layout()
    cm_path = os.path.join(output_dir, "confusion_matrix.png")
    fig.savefig(cm_path, bbox_inches="tight")
    plt.close(fig)
    print(f"[+] Saved confusion matrix: {cm_path}")

    # 3. Performance Metrics Bar Chart
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, pos_label="spam")
    rec = recall_score(y_test, y_pred, pos_label="spam")
    f1 = f1_score(y_test, y_pred, pos_label="spam")

    metric_names = ["Accuracy", "Precision", "Recall", "F1-Score"]
    metric_values = [acc * 100, prec * 100, rec * 100, f1 * 100]
    metric_colors = ["#1f77b4", "#2ca02c", "#ff7f0e", "#9467bd"]

    fig, ax = plt.subplots(figsize=(6, 4), dpi=300)
    bars = ax.bar(metric_names, metric_values, color=metric_colors, width=0.55, edgecolor="black",
        linewidth=1.1)
    ax.set_ylim(0, 115)
    ax.set_ylabel("Score (%)", fontsize=10, fontweight="bold")
    ax.set_title("Model Performance Metrics (Spam Class)", fontsize=12, fontweight="bold", pad=12)
    ax.grid(axis="y", linestyle="--", alpha=0.5)

    for bar in bars:
        h = bar.get_height()
        ax.annotate(
            f"{h:.2f}%",
            xy=(bar.get_x() + bar.get_width() / 2, h),
```

```
            xytext=(0, 4),
            textcoords="offset points",
            ha="center",
            va="bottom",
            fontsize=9.5,
            fontweight="bold",
        )
    plt.tight_layout()
    metrics_path = os.path.join(output_dir, "metrics_chart.png")
    fig.savefig(metrics_path, bbox_inches="tight")
    plt.close(fig)
    print(f"[+] Saved metrics chart: {metrics_path}")


# ============================================================================
# 4. MAIN PIPELINE EXECUTION
# ============================================================================
def main():
    base_dir = os.path.dirname(os.path.abspath(__file__))
    project_root = os.path.abspath(os.path.join(base_dir, ".."))
    dataset_path = os.path.join(project_root, "dataset", "SMSSpamCollection.txt")
    screenshots_dir = os.path.join(project_root, "submission_report", "screenshots")

    print("=" * 70)
    print(" AI 484: SMS Spam Detection using Naive Bayes Classifier")
    print("=" * 70)

    # 1. Load Dataset
    print("\n[Step 1] Loading SMS Spam Collection dataset...")
    df = load_dataset(dataset_path)
    print(f"Total rows: {len(df)}")
    print(f"Class distribution:\n{df['label'].value_counts()}")

    # 2. Preprocess Text
    print("\n[Step 2] Preprocessing message texts...")
    df["cleaned_message"] = df["message"].apply(clean_text)

    # 3. Train-Test Split (80% Train, 20% Test with Stratification)
    print("\n[Step 3] Splitting dataset into train (80%) and test (20%)...")
    X_train, X_test, y_train, y_test = train_test_split(
        df["cleaned_message"],
        df["label"],
        test_size=0.20,
        random_state=42,
        stratify=df["label"],
    )
    print(f"Train samples: {len(X_train)} | Test samples: {len(X_test)}")

    # 4. Vectorization (CountVectorizer / Bag-of-Words)
    print("\n[Step 4] Vectorizing text with CountVectorizer (Bag of Words)...")
    vectorizer = CountVectorizer()
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)
    print(f"Vocabulary size: {len(vectorizer.vocabulary_)} distinct tokens")

    # 5. Train Naive Bayes Classifier
    print("\n[Step 5] Training Multinomial Naive Bayes classifier...")
    classifier = MultinomialNB(alpha=1.0)
    classifier.fit(X_train_vec, y_train)
    print("Model training completed successfully.")

    # 6. Evaluation on Unseen Test Set
    print("\n[Step 6] Evaluating model on unseen test set...")
    y_pred = classifier.predict(X_test_vec)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, pos_label="spam")
    rec = recall_score(y_test, y_pred, pos_label="spam")
    f1 = f1_score(y_test, y_pred, pos_label="spam")

    print("\n" + "=" * 50)
    print(" EVALUATION METRICS:")
    print("=" * 50)
    print(f" Accuracy : {acc * 100:.2f}%")
    print(f" Precision: {prec * 100:.2f}% (Spam)")
    print(f" Recall   : {rec * 100:.2f}% (Spam)")
    print(f" F1-Score : {f1 * 100:.2f}% (Spam)")
    print("\nDetailed Classification Report:")
    print(classification_report(y_test, y_pred, target_names=["ham", "spam"]))
```

```
227
228     # 7. Predict on Required Test Query
229     print("=" * 50)
230     print(" INFERENCE ON REQUIRED LAB TEST QUERY:")
231     print("=" * 50)
232     test_query = "Congratulations! You have won a free ticket. Call now!"
233     cleaned_query = clean_text(test_query)
234     query_vec = vectorizer.transform([cleaned_query])
235     prediction = classifier.predict(query_vec)[0]
236     probabilities = classifier.predict_proba(query_vec)[0]
237     class_indices = {cls: idx for idx, cls in enumerate(classifier.classes_)}
238
239     print(f"Input Message   : \"{test_query}\"")
240     print(f"Cleaned Message : \"{cleaned_query}\"")
241     print(f"Predicted Class : {prediction.upper()}")
242     print(f"Probability [Ham]  : {probabilities[class_indices['ham']] * 100:.4f}%")
243     print(f"Probability [Spam] : {probabilities[class_indices['spam']] * 100:.4f}%")
244
245     # 8. Export Visualizations for LaTeX Report
246     print("\n[Step 7] Exporting visualizations for LaTeX report...")
247     export_visualizations(df, y_test, y_pred, screenshots_dir)
248     print("\nExecution finished successfully!")
249
250
251 if __name__ == "__main__":
252     main()
```